

# **COURSE MATERIAL**

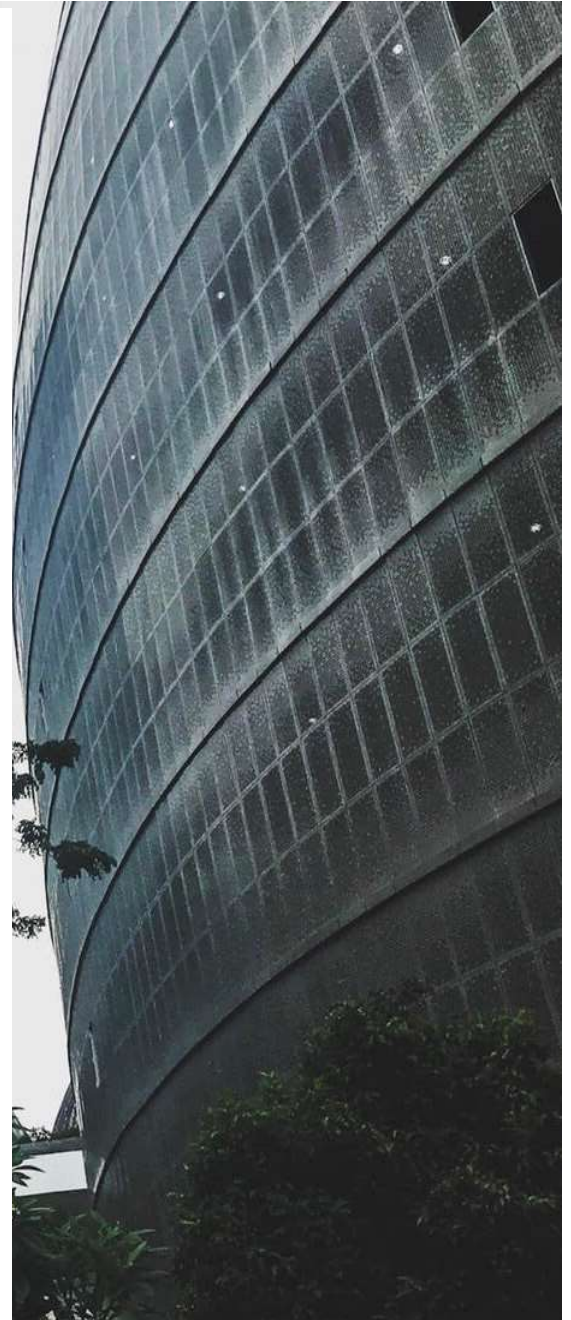
**IF670 – Mobile Cross Platform Programming  
S1 INFORMATICS GRADUATE PROGRAM  
FACULTY OF ENGINEERING AND INFORMATICS**

---

---

**INFORMATICS STUDY PROGRAM  
FACULTY OF ENGINEERING AND INFORMATION  
UNIVERSITAS MULTIMEDIA NUSANTARA**

Gedung B Lantai 5, Kampus UMN  
Jl. Scientia Boulevard, Gading Serpong, Tangerang-15811 Indonesia  
Telp: +62-21.5422.0808 (ext. 1803), email: [lab.fti@umn.ac.id](mailto:lab.fti@umn.ac.id), web: [umn.ac.id](http://umn.ac.id)



# MODUL 12

## (Notifications)

### THEME DESCRIPTION

In Week 12, students learn about the concept of push notifications in modern mobile applications. Push notifications are a mechanism for sending messages from a server to a user's device, even when the app is not currently active on the screen. This feature is commonly used in instant messaging apps, e-commerce platforms, academic systems, transport apps, financial apps and public service apps to notify users of important activities in real time.

### WEEKLY LEARNING GOAL (SUB-GOAL LEARNING)

Upon completing Module 12, students are expected to be able to:

1. Explain the basic concepts of push notifications in mobile applications.
2. Explain the role of Firebase Cloud Messaging in sending Android notifications.
3. Explain the relationship between Expo Notifications, Expo Push Tokens and FCM.

### SOFTWARE / HARDWARE REQUIREMENTS

1. Laptop / PC
2. Android Studio / physical Android device
3. Code editor (Visual Studio Code/Sublime/etc.)
4. Node.js and npm
5. Expo project
6. Firebase / Google Account
7. Expo Go app / Development build

### PRACTICUM STEPS

1. Create a new Expo project. Then install the necessary dependencies:



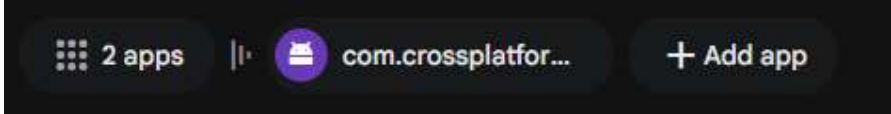
```
1 npx expo install expo-notifications expo-constants eas-cli
```

2. Creating a Firebase project

1. Open the Firebase Console.
2. Select 'Add project' or 'Create project'.
3. Name the project, for example: IF670 Week12 Notifications
4. Follow the project creation process until completion.

3. Adding an Android app to Firebase

On the Firebase project page:

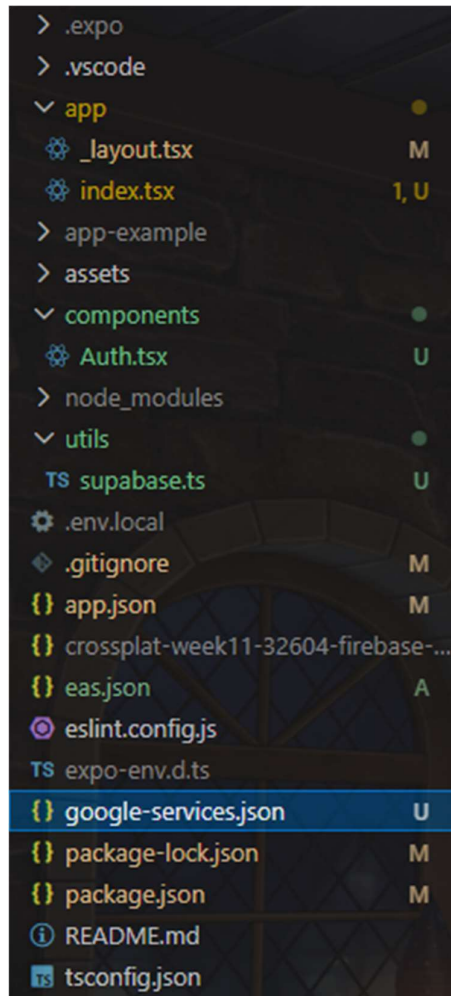
- 
- 1.
  2. Click the Add app button, then click the Android icon to add an Android app.
  3. Enter the Android package name. Example: com.if670.week12notifications
  4. Then click the Register app button

**Note:** The package name must match the Android configuration in the Expo project. The package name serves as the Android app's identifier on devices and in the Google Play Store.

#### 4. Download the google-services.json file

Once the Android app has been successfully registered:

1. Click Download google-services.json.
2. Save the file to the root of your Expo project.



#### 5. Configuring app.json

1. Open the app.json file, then adjust the following configuration.

```

1  {
2    "expo": {
3      ...
4      "android": {
5        ...
6        "googleServicesFile": "./google-services.json",
7        "package": "com.crossplatform.umn"
8        ...
9      },
10     "plugins": [
11       ...
12       "expo-notifications"
13       ...
14     ],
15   }
16 }

```

Note:

android.package - Must match the Android package name entered in Firebase.

android.googleServicesFile - Specifies the location of the google-services.json file.

plugins

plugin expo-notifications - so that the necessary native configurations can be set up during the build.

6. Add the following code to index.tsx

```

1  import { useState, useEffect } from 'react';
2  import { Text, View, Button, Platform } from 'react-native';
3  import * as Notifications from 'expo-notifications';
4  import Constants from 'expo-constants';
5
6
7  Notifications.setNotificationHandler({
8    handleNotification: async () => ({
9      shouldPlaySound: true,
10     shouldSetBadge: true,
11     shouldShowBanner: true,
12     shouldShowList: true,
13   }),
14 });
15
16
17
18  async function sendPushNotification(expoPushToken: string) {
19    const message = {
20      to: expoPushToken,
21      sound: 'default',
22      title: 'Original Title',
23      body: 'And here is the body!',
24      data: { someData: 'goes here' },
25    };
26
27    await fetch('https://exp.host/--/api/v2/push/send', {
28      method: 'POST',
29      headers: {
30        Accept: 'application/json',
31        'Accept-encoding': 'gzip, deflate',
32        'Content-Type': 'application/json',
33      },
34      body: JSON.stringify(message),
35    });
36  }
37

```

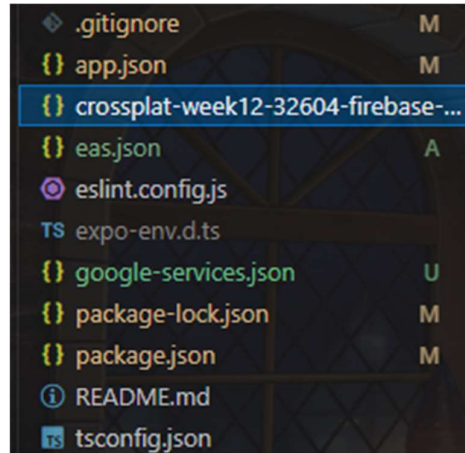
```

39 function handleRegistrationError(errorMessage: string) {
40   alert(errorMessage);
41   throw new Error(errorMessage);
42 }
43
44 async function registerForPushNotificationsAsync() {
45   if (Platform.OS === 'android') {
46     await Notifications.setNotificationChannelAsync('default', {
47       name: 'default',
48       importance: Notifications.AndroidImportance.MAX,
49       vibrationPattern: [0, 250, 250, 250],
50       lightColor: '#FF231F7C',
51     });
52   }
53
54   const { status: existingStatus } = await Notifications.getPermissionsAsync();
55   let finalStatus = existingStatus;
56   if (existingStatus !== 'granted') {
57     const { status } = await Notifications.requestPermissionsAsync();
58     finalStatus = status;
59   }
60   if (finalStatus !== 'granted') {
61     handleRegistrationError('Permission not granted to get push token for push notification!');
62     return;
63   }
64   const projectId = Constants?.expoConfig?.extra?.eas?.projectId ?? Constants?.easConfig?.projectId;
65   if (!projectId) {
66     handleRegistrationError('Project ID not found');
67   }
68   try {
69     const pushTokenString = (
70       await Notifications.getExpoPushTokenAsync({
71         projectId,
72       })
73     ).data;
74     console.log(pushTokenString);
75     return pushTokenString;
76   } catch (e: unknown) {
77     handleRegistrationError(`${e}`);
78   }
79 }
80
81 export default function App() {
82   const [expoPushToken, setExpoPushToken] = useState('');
83   const [notification, setNotification] = useState<Notifications.Notification | undefined>(
84     undefined
85   );
86
87   useEffect(() => {
88     registerForPushNotificationsAsync()
89       .then(token => setExpoPushToken(token ?? ''))
90       .catch((error: any) => setExpoPushToken(`${error}`));
91
92     const notificationListener = Notifications.addNotificationReceivedListener(notification => {
93       setNotification(notification);
94     });
95
96     const responseListener = Notifications.addNotificationResponseReceivedListener(response => {
97       console.log(response);
98     });
99
100    return () => {
101      notificationListener.remove();
102      responseListener.remove();
103    };
104  }, []);
105
106   return (
107     <View style={{ flex: 1, alignItems: 'center', justifyContent: 'space-around' }}>
108       <Text>Your Expo push token: {expoPushToken}</Text>
109       <View style={{ alignItems: 'center', justifyContent: 'center' }}>
110         <Text>Title: {notification && notification.request.content.title}</Text>
111         <Text>Body: {notification && notification.request.content.body}</Text>
112         <Text>Data: {notification && JSON.stringify(notification.request.content.data)}</Text>
113       </View>
114       <Button
115         title="Press to Send Notification"
116         onPress={async () => {
117           await sendPushNotification(expoPushToken);
118         }}
119       />
120     </View>
121   );
122 }
123

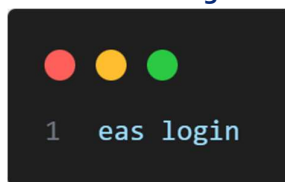
```

## 7. Setting up Firebase Cloud Messaging

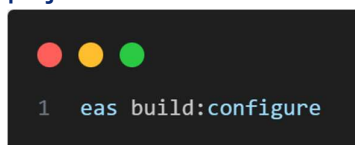
1. On the Firebase console page, open Project Settings, then select the Service accounts tab
2. Click Generate New Private Key, and confirm by clicking Generate Key. Save the JSON file containing the private key to your project root.



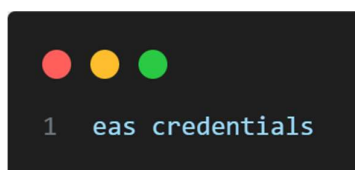
3. Add the JSON file to .gitignore so that it is not included when you commit to your repository, as it contains sensitive data.
8. Create an Expo account to integrate the Expo Application Service (EAS). You can register for an account via the following link: <https://expo.dev/login>
9. Run the following command in your terminal to log in again via the terminal.



10. Run the following command in the terminal and ensure that the eas.json file appears in your project.



11. Run the following command in your terminal to start configuring EAS



1. Select Android > Development > Google Service Account
2. Select Manage your Google Service Account Key for Push Notifications (FCM V1)
3. Select Set up a Google Service Account Key for Push Notifications (FCM V1) > Upload a new service account key
4. If you have previously saved a JSON file in your project directory, the EAS CLI will automatically detect the JSON file and prompt you to select it. Press Y to continue.
12. Run the following command in your terminal. Ensure that an eas.json file has been generated in your project and that the project you have built appears in the dashboard on your expo.dev page once the command has been run.





```
1 eas build --profile development --platform android
```

Note: Please wait a few moments for the build to complete.

13. Please start the development server for your project using the following command



```
1 npx expo start --dev-client
```

14. Click the 'Press to Send Notification' button to check whether the notification setup has been successful.

## ASSIGNMENT

Modify your Week 11 assignment. Whenever data is inserted into Supabase, add a notification indicating whether the data was successfully inserted or failed to be inserted into the database. Also include the longitude and latitude in your notification.

## REFERENCES

<https://docs.expo.dev/push-notifications/fcm-credentials/>

<https://docs.expo.dev/push-notifications/push-notifications-setup/#android>

<https://docs.expo.dev/versions/latest/sdk/notifications/>